

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from collections import defaultdict

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (accuracy_score, precision_score, re
                             f1_score, confusion_matrix, classif
                             roc_auc_score, roc_curve)

import warnings
warnings.filterwarnings('ignore')

from google.colab import drive
drive.mount('/content/drive')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"✅ Libraries imported")
print(f"Device: {device}")
if device.type == 'cuda':
    print(f"GPU: {torch.cuda.get_device_name(0)}")

```

Mounted at /content/drive

✅ Libraries imported  
Device: cuda  
GPU: Tesla T4

```

from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```

# =====
# الخلية 2: تحميل الصور وتنظيمها
# =====

```

```

# مسارات المجلدات في Google Drive
BASE = '/content/drive/MyDrive/'

folders = {
    'chanel_fake': os.path.join(BASE, '/content/drive/MyDrive/GF'),
    'chanel_real': os.path.join(BASE, '/content/drive/MyDrive/GF'),
    'coach_fake': os.path.join(BASE, '/content/drive/MyDrive/GF'),
    'coach_real': os.path.join(BASE, '/content/drive/MyDrive/GF'),
    # — Fendi —
    'fendi_fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',
    'fendi_real': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',

    'lv_fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',
    'lv_real': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',

    'prada_fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',
    'prada_real': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',

    'ysl_fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',
    'ysl_real': '/content/drive/MyDrive/GP2 (أصيل)/Data/prod',
}

def get_brand(key):
    mapping = {
        'chanel': 'Chanel',
        'coach': 'Coach',
        'fendi': 'Fendi',
        'lv': 'LV',
        'prada': 'Prada',
        'ysl': 'YSL',
    }
    for k, v in mapping.items():
        if key.startswith(k):
            return v
    return key

image_data = []

for key, folder_path in folders.items():
    if folder_path == '':
        print(f"⚠️ مسار فاضي: {key} - تخطي")
        continue

    brand = get_brand(key)
    is_authentic = 1 if 'real' in key else 0

```

```

if not os.path.exists(folder_path):
    print(f"⚠ Folder not found: {folder_path}")
    continue

files = sorted([f for f in os.listdir(folder_path)
                 if f.lower().endswith(('.jpg', '.jpeg', '.png'))])

for f in files:
    filepath = os.path.join(folder_path, f)
    parts = f.replace('.jpg', '').replace('.jpeg', '').replace('.png', '')
    product_id = None
    for p in parts:
        try:
            num = int(p)
            if product_id is None:
                product_id = num
        except:
            continue
    if product_id is None:
        product_id = f

    image_data.append({
        'filepath':    filepath,
        'filename':    f,
        'product_id':  f"{brand}_{is_authentic}_{product_id}",
        'brand':       brand,
        'is_authentic': is_authentic,
    })

print(f"✅ {key}: {len(files)} images")

df_images = pd.DataFrame(image_data)

product_groups = df_images.groupby('product_id').agg({
    'filepath':    list,
    'is_authentic': 'first',
    'brand':       'first',
}).reset_index()

print(f"\n📊 Summary:")
print(f"    Total images:    {len(df_images)}")
print(f"    Total products: {len(product_groups)}")
print(f"    Fake products:   {len(product_groups[product_groups['is_authentic'] == False])}")
print(f"    Real products:   {len(product_groups[product_groups['is_authentic'] == True])}")
if len(product_groups) > 0:
    print(f"    Avg. imgs/product: {len(df_images)/len(product_groups)}")

```

```

print(f"    Avg imgs/product: {len(imgs)/len(product_groups)}")

print(f"\n📦 Per-brand breakdown:")
for brand in product_groups['brand'].unique():
    sub = product_groups[product_groups['brand'] == brand]
    fake = len(sub[sub['is_authentic']==0])
    real = len(sub[sub['is_authentic']==1])
    print(f"    {brand:<10}: fake={fake}    real={real}")

```

```

✓ chanel_fake: 362 images
✓ chanel_real: 517 images
✓ coach_fake: 388 images
✓ coach_real: 559 images
✓ fendi_fake: 305 images
✓ fendi_real: 417 images
✓ lv_fake: 54 images
✓ lv_real: 396 images
✓ prada_fake: 527 images
✓ prada_real: 509 images
✓ ysl_fake: 592 images
✓ ysl_real: 410 images

```

 Summary:

```

Total images:    5036
Total products:  1923
Fake products:   944
Real products:   979
Avg imgs/product: 2.6

```

 Per-brand breakdown:

```

Chanel    : fake=94    real=94
Coach     : fake=94    real=94
Fendi     : fake=86    real=93
LV        : fake=47    real=94
Prada     : fake=527   real=509
YSL       : fake=96    real=95

```

```

# =====
# 3 الخلية : Dataset Class & Transforms
# =====

class BagImageDataset(Dataset):
    def __init__(self, product_ids, labels, image_paths_dict,
                  self.product_ids      = product_ids
                  self.labels           = labels
                  self.image_paths_dict = image_paths_dict
                  self.transform        = transform

    def __len__(self):
        return len(self.product_ids)

```

```
def __getitem__(self, idx):
    pid    = self.product_ids[idx]
    label  = self.labels[idx]

    paths   = self.image_paths_dict[pid]
    img_path = paths[np.random.randint(len(paths))]

    try:
        image = Image.open(img_path).convert('RGB')
    except:
        image = Image.new('RGB', (224, 224), (0, 0, 0))

    if self.transform:
        image = self.transform(image)

    return image, torch.tensor(label, dtype=torch.float32)

train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

test_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

print("✅ Dataset class & transforms defined")
```

✅ Dataset class & transforms defined

```
# =====
# الخلية 4: بناء EfficientNet-B0
# =====

def create_efficientnet():
    model = models.efficientnet_b0(weights=models.EfficientNet_

    for param in model.parameters():
        param.requires_grad = False

    for param in model.features[-3:].parameters():
        param.requires_grad = True

    model.classifier = nn.Sequential(
        nn.Dropout(0.4),
        nn.Linear(1280, 256),
        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(256, 1),
        nn.Sigmoid()
    )

    return model

model_test = create_efficientnet()
trainable = sum(p.numel() for p in model_test.parameters() if
total = sum(p.numel() for p in model_test.parameters())
print(f"✅ EfficientNet-B0 created:")
print(f"    Total parameters:    {total:,}")
print(f"    Trainable parameters: {trainable:,} ({trainable/total*100:.1f}%)")
print(f"    Frozen parameters:    {total-trainable:,}")
del model_test
```

```
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rw
100%|██████████| 20.5M/20.5M [00:00<00:00, 159MB/s]✅ EfficientNet-B0
Total parameters:    4,335,741
Trainable parameters: 3,483,933 (80.4%)
Frozen parameters:    851,808
```

```
# =====
# الخلية 5: دوال التدريب والتقييم
# =====

def train_image_model(model, train_loader, val_loader, epochs=
```

```

criterion = nn.BCELoss()
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad_(), model.parameters()), lr=lr, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=10)

history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
best_val_loss = float('inf')
best_state = None
patience_counter = 0

for epoch in range(epochs):
    # — Train —
    model.train()
    t_loss, t_correct, t_total = 0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images).squeeze()
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        t_loss += loss.item()
        t_correct += ((outputs >= 0.5).float() == labels).sum().item()
        t_total += len(labels)

    # — Validation —
    model.eval()
    v_loss, v_correct, v_total = 0, 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images).squeeze()
            v_loss += criterion(outputs, labels).item()
            v_correct += ((outputs >= 0.5).float() == labels).sum().item()
            v_total += len(labels)

    avg_tl = t_loss / len(train_loader)
    avg_vl = v_loss / len(val_loader)
    t_acc = t_correct / t_total
    v_acc = v_correct / v_total

    history['train_loss'].append(avg_tl)
    history['val_loss'].append(avg_vl)
    history['train_acc'].append(t_acc)
    history['val_acc'].append(v_acc)

```

```

scheduler.step(avg_vl)

marker = ""
if avg_vl < best_val_loss:
    best_val_loss = avg_vl
    best_state = {k: v.cpu().clone() for k, v in
    patience_counter = 0
    marker = "✅"
else:
    patience_counter += 1

print(f"    Epoch {epoch+1:>2}/{epochs} | TrLoss:{avg_t

if patience_counter >= patience:
    print(f"    🛑 Early stopping at epoch {epoch+1}")
    break

model.load_state_dict(best_state)
model.to(device)
return model, history

def evaluate_image_model(model, test_loader):
    model.eval()
    preds, probs, labels = [], [], []
    with torch.no_grad():
        for images, lbls in test_loader:
            images = images.to(device)
            outputs = model(images).squeeze()
            preds.extend((outputs >= 0.5).float().cpu().numpy(
            probs.extend(outputs.cpu().numpy())
            labels.extend(lbls.numpy())
    return np.array(preds), np.array(probs), np.array(labels)

print("✅ Training functions defined")

```

✅ Training functions defined

```

# The corrected evaluate_image_model function
def evaluate_image_model(model, test_loader):
    model.eval()
    preds, probs, labels = [], [], []
    with torch.no_grad():
        for images, lbls in test_loader:
            images = images.to(device)

```



```

        outputs = model(images).squeeze()
        # Ensure the numpy array is 1D before extending to
        preds.extend((outputs >= 0.5).float().cpu().numpy())
        probs.extend(outputs.cpu().numpy().flatten())
        labels.extend(lbls.numpy()) # lbls is already numpy
    return np.array(preds), np.array(probs), np.array(labels)

# =====
# 5 :6 -Fold Cross-Validation الخلية
# =====

N_FOLDS      = 5
EPOCHS       = 30
BATCH_SIZE   = 32 # يتحمل GPU زيادة من 16 - بيانات أكبر والـ

product_ids   = product_groups['product_id'].values
product_labels = product_groups['is_authentic'].values
product_brands = product_groups['brand'].values
stratify_col  = [f"{b}_{l}" for b, l in zip(product_brands, product_labels)]

image_paths_dict = dict(zip(product_groups['product_id'],
                             product_groups['filepath']))

skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=42)

all_fold_results = []
all_true         = []
all_pred         = []
all_prob         = []
fold_histories   = []

print(f"🚀 Starting {N_FOLDS}-Fold Cross-Validation")
print(f"   Products: {len(product_ids)} | Images: {len(df_image_paths)}")
print(f"   Model: EfficientNet-B0 | Epochs: {EPOCHS} | Batch: {BATCH_SIZE}")
print(f"   { '='*70 })

for fold, (train_idx, test_idx) in enumerate(skf.split(product_ids, stratify_col)):
    print(f"\n{'='*60}")
    print(f"📌 FOLD {fold+1}/{N_FOLDS}")
    print(f"{'='*60}")

    train_pids   = product_ids[train_idx]
    test_pids    = product_ids[test_idx]
    train_labels = product_labels[train_idx]
    test_labels  = product_labels[test_idx]

```

```

val_size          = int(0.15 * len(train_pids))
val_pids          = train_pids[-val_size:]
val_labels_split  = train_labels[-val_size:]
train_pids_split  = train_pids[:-val_size]
train_labels_split = train_labels[:-val_size]

print(f"    Train: {len(train_pids_split)} products")
print(f"    Val:   {len(val_pids)} products")
print(f"    Test:  {len(test_pids)} products")

train_ds = BagImageDataset(train_pids_split, train_labels_split)
val_ds   = BagImageDataset(val_pids,        val_labels_split)
test_ds  = BagImageDataset(test_pids,        test_labels,

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE,
test_loader  = DataLoader(test_ds,  batch_size=BATCH_SIZE,

model = create_efficientnet().to(device)
model, history = train_image_model(model, train_loader, val_loader)
fold_histories.append(history)

fold_probs_list = []
for _ in range(5):
    _, probs, _ = evaluate_image_model(model, test_loader)
    fold_probs_list.append(probs)

avg_probs = np.mean(fold_probs_list, axis=0)
final_preds = (avg_probs >= 0.5).astype(float)

acc = accuracy_score(test_labels, final_preds)
prec = precision_score(test_labels, final_preds, zero_division=0)
rec = recall_score(test_labels, final_preds, zero_division=0)
f1 = f1_score(test_labels, final_preds, zero_division=0)
auc = roc_auc_score(test_labels, avg_probs)

all_fold_results.append({'Fold': fold+1, 'Accuracy': acc,
                        'Recall': rec, 'F1': f1, 'AUC': auc})
all_true.extend(test_labels)
all_pred.extend(final_preds)
all_prob.extend(avg_probs)

print(f"\n  Fold {fold+1} Results: Acc={acc:.3f} Prec

```

```

del model
torch.cuda.empty_cache() if device.type == 'cuda' else None

all_true = np.array(all_true)
all_pred = np.array(all_pred)
all_prob = np.array(all_prob)

print(f"\n{'='*70}")
print("✅ All folds completed!")

```

|             |               |               |             |         |
|-------------|---------------|---------------|-------------|---------|
| Epoch 17/30 | TrLoss:0.0186 | VLLoss:0.1394 | TrAcc:0.995 | VlAcc:0 |
| Epoch 18/30 | TrLoss:0.0343 | VLLoss:0.0541 | TrAcc:0.990 | VlAcc:0 |
| Epoch 19/30 | TrLoss:0.0362 | VLLoss:0.1409 | TrAcc:0.991 | VlAcc:0 |
| Epoch 20/30 | TrLoss:0.0273 | VLLoss:0.1170 | TrAcc:0.992 | VlAcc:0 |
| Epoch 21/30 | TrLoss:0.0337 | VLLoss:0.0722 | TrAcc:0.988 | VlAcc:0 |
| Epoch 22/30 | TrLoss:0.0408 | VLLoss:0.0984 | TrAcc:0.985 | VlAcc:0 |
| Epoch 23/30 | TrLoss:0.0373 | VLLoss:0.0756 | TrAcc:0.985 | VlAcc:0 |
| Epoch 24/30 | TrLoss:0.0300 | VLLoss:0.1234 | TrAcc:0.991 | VlAcc:0 |
| Epoch 25/30 | TrLoss:0.0422 | VLLoss:0.0533 | TrAcc:0.988 | VlAcc:0 |
| Epoch 26/30 | TrLoss:0.0288 | VLLoss:0.0982 | TrAcc:0.992 | VlAcc:0 |
| Epoch 27/30 | TrLoss:0.0256 | VLLoss:0.0870 | TrAcc:0.993 | VlAcc:0 |
| Epoch 28/30 | TrLoss:0.0326 | VLLoss:0.1442 | TrAcc:0.990 | VlAcc:0 |
| Epoch 29/30 | TrLoss:0.0195 | VLLoss:0.1355 | TrAcc:0.994 | VlAcc:0 |
| Epoch 30/30 | TrLoss:0.0254 | VLLoss:0.1231 | TrAcc:0.993 | VlAcc:0 |



Fold 1 Results: Acc=0.977 Prec=0.961 Rec=0.995 F1=0.977



FOLD 2/5

Train: 1308 products

Val: 230 products

Test: 385 products

|             |               |               |             |         |
|-------------|---------------|---------------|-------------|---------|
| Epoch 1/30  | TrLoss:0.5793 | VLLoss:0.4795 | TrAcc:0.678 | VlAcc:0 |
| Epoch 2/30  | TrLoss:0.3572 | VLLoss:0.2929 | TrAcc:0.886 | VlAcc:0 |
| Epoch 3/30  | TrLoss:0.2208 | VLLoss:0.1506 | TrAcc:0.931 | VlAcc:0 |
| Epoch 4/30  | TrLoss:0.1615 | VLLoss:0.1298 | TrAcc:0.941 | VlAcc:0 |
| Epoch 5/30  | TrLoss:0.1370 | VLLoss:0.0935 | TrAcc:0.948 | VlAcc:0 |
| Epoch 6/30  | TrLoss:0.1070 | VLLoss:0.0803 | TrAcc:0.965 | VlAcc:0 |
| Epoch 7/30  | TrLoss:0.0761 | VLLoss:0.1006 | TrAcc:0.973 | VlAcc:0 |
| Epoch 8/30  | TrLoss:0.1019 | VLLoss:0.1094 | TrAcc:0.970 | VlAcc:0 |
| Epoch 9/30  | TrLoss:0.0693 | VLLoss:0.0954 | TrAcc:0.980 | VlAcc:0 |
| Epoch 10/30 | TrLoss:0.0541 | VLLoss:0.0644 | TrAcc:0.984 | VlAcc:0 |
| Epoch 11/30 | TrLoss:0.0844 | VLLoss:0.0992 | TrAcc:0.966 | VlAcc:0 |
| Epoch 12/30 | TrLoss:0.0559 | VLLoss:0.0887 | TrAcc:0.983 | VlAcc:0 |
| Epoch 13/30 | TrLoss:0.0375 | VLLoss:0.0736 | TrAcc:0.987 | VlAcc:0 |
| Epoch 14/30 | TrLoss:0.0374 | VLLoss:0.0966 | TrAcc:0.990 | VlAcc:0 |
| Epoch 15/30 | TrLoss:0.0354 | VLLoss:0.1462 | TrAcc:0.990 | VlAcc:0 |
| Epoch 16/30 | TrLoss:0.0322 | VLLoss:0.0746 | TrAcc:0.988 | VlAcc:0 |
| Epoch 17/30 | TrLoss:0.0322 | VLLoss:0.0741 | TrAcc:0.988 | VlAcc:0 |
| Epoch 18/30 | TrLoss:0.0324 | VLLoss:0.0438 | TrAcc:0.992 | VlAcc:0 |
| Epoch 19/30 | TrLoss:0.0385 | VLLoss:0.1142 | TrAcc:0.988 | VlAcc:0 |

|             |  |               |               |  |             |         |
|-------------|--|---------------|---------------|--|-------------|---------|
| Epoch 20/30 |  | TrLoss:0.0364 | VlLoss:0.0691 |  | TrAcc:0.987 | VlAcc:0 |
| Epoch 21/30 |  | TrLoss:0.0355 | VlLoss:0.0231 |  | TrAcc:0.987 | VlAcc:0 |
| Epoch 22/30 |  | TrLoss:0.0233 | VlLoss:0.0950 |  | TrAcc:0.991 | VlAcc:0 |
| Epoch 23/30 |  | TrLoss:0.0286 | VlLoss:0.1185 |  | TrAcc:0.993 | VlAcc:0 |
| Epoch 24/30 |  | TrLoss:0.0273 | VlLoss:0.0897 |  | TrAcc:0.996 | VlAcc:0 |
| Epoch 25/30 |  | TrLoss:0.0273 | VlLoss:0.1027 |  | TrAcc:0.991 | VlAcc:0 |
| Epoch 26/30 |  | TrLoss:0.0174 | VlLoss:0.0904 |  | TrAcc:0.995 | VlAcc:0 |
| Epoch 27/30 |  | TrLoss:0.0282 | VlLoss:0.1190 |  | TrAcc:0.991 | VlAcc:0 |
| Epoch 28/30 |  | TrLoss:0.0257 | VlLoss:0.1239 |  | TrAcc:0.992 | VlAcc:0 |
| Epoch 29/30 |  | TrLoss:0.0356 | VlLoss:0.1109 |  | TrAcc:0.990 | VlAcc:0 |
| Epoch 30/30 |  | TrLoss:0.0312 | VlLoss:0.1174 |  | TrAcc:0.993 | VlAcc:0 |



Fold 2 Results: Acc=0.990 Prec=0.980 Rec=1.000 F1=0.990



FOLD 3/5

```
# =====
# الخلية 7: جدول النتائج
# =====

fold_df = pd.DataFrame(all_fold_results)

print("\n" + "="*80)
print("🇮🇹 EXPERIMENT 2: IMAGE ONLY – RESULTS (5-Fold CV)")
print("="*80)
print(f"\n{'Fold':>6} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1-Score':>10} {'AUC-ROC':>10}")
print("-" * 60)
for _, row in fold_df.iterrows():
    print(f"{int(row['Fold']):>6} {row['Accuracy']:>10.4f} {row['Precision']:>10.4f} {row['Recall']:>10.4f} {row['F1-Score']:>10.4f} {row['AUC-ROC']:>10.4f}")
print("-" * 60)
print(f"{'Mean':>6} {fold_df['Accuracy'].mean():>10.4f} {fold_df['Precision'].mean():>10.4f} {fold_df['Recall'].mean():>10.4f} {fold_df['F1-Score'].mean():>10.4f} {fold_df['AUC-ROC'].mean():>10.4f}")
print(f"{'Std':>6} {fold_df['Accuracy'].std():>10.4f} {fold_df['Precision'].std():>10.4f} {fold_df['Recall'].std():>10.4f} {fold_df['F1-Score'].std():>10.4f} {fold_df['AUC-ROC'].std():>10.4f}")
print(f"\n🇮🇹 Final: Acc={fold_df['Accuracy'].mean():.4f}±{fold_df['Accuracy'].std():.4f} F1={fold_df['F1-Score'].mean():.4f}±{fold_df['F1-Score'].std():.4f} AUC={fold_df['AUC-ROC'].mean():.4f}±{fold_df['AUC-ROC'].std():.4f}")
display(fold_df)
```

```
=====
🇮🇹 EXPERIMENT 2: IMAGE ONLY – RESULTS (5-Fold CV)
=====
```

| Fold | Accuracy | Precision | Recall | F1-Score | AUC-ROC |
|------|----------|-----------|--------|----------|---------|
| 1    | 0.9766   | 0.9606    | 0.9949 | 0.9774   | 0.9959  |
| 2    | 0.9896   | 0.9798    | 1.0000 | 0.9898   | 0.9994  |
| 3    | 0.9610   | 0.9372    | 0.9898 | 0.9628   | 0.9869  |
| 4    | 0.9792   | 0.9653    | 0.9949 | 0.9799   | 0.9983  |
| 5    | 0.9740   | 0.9517    | 1.0000 | 0.9752   | 0.9928  |
| Mean | 0.9761   | 0.9589    | 0.9959 | 0.9770   | 0.9947  |
| Std  | 0.0103   | 0.0158    | 0.0043 | 0.0097   | 0.0050  |

```
🇮🇹 Final: Acc=0.9761±0.0103 F1=0.9770±0.0097 AUC=0.9947±0.0050
```

|   | Fold | Accuracy | Precision | Recall   | F1       | AUC      |
|---|------|----------|-----------|----------|----------|----------|
| 0 | 1    | 0.976623 | 0.960591  | 0.994898 | 0.977444 | 0.995924 |
| 1 | 2    | 0.989610 | 0.979798  | 1.000000 | 0.989796 | 0.999406 |
| 2 | 3    | 0.961039 | 0.937198  | 0.989796 | 0.962779 | 0.986934 |
| 3 | 4    | 0.979167 | 0.965347  | 0.994898 | 0.979899 | 0.998290 |
| 4 | 5    | 0.973958 | 0.951691  | 1.000000 | 0.975248 | 0.992779 |

```
# =====
```

## # 8 الخلية : Training History Plot

#

```

fig, axes = plt.subplots(2, 5, figsize=(20, 8))
fig.suptitle('Experiment 2: Training History per Fold – EfficientNet-B0')

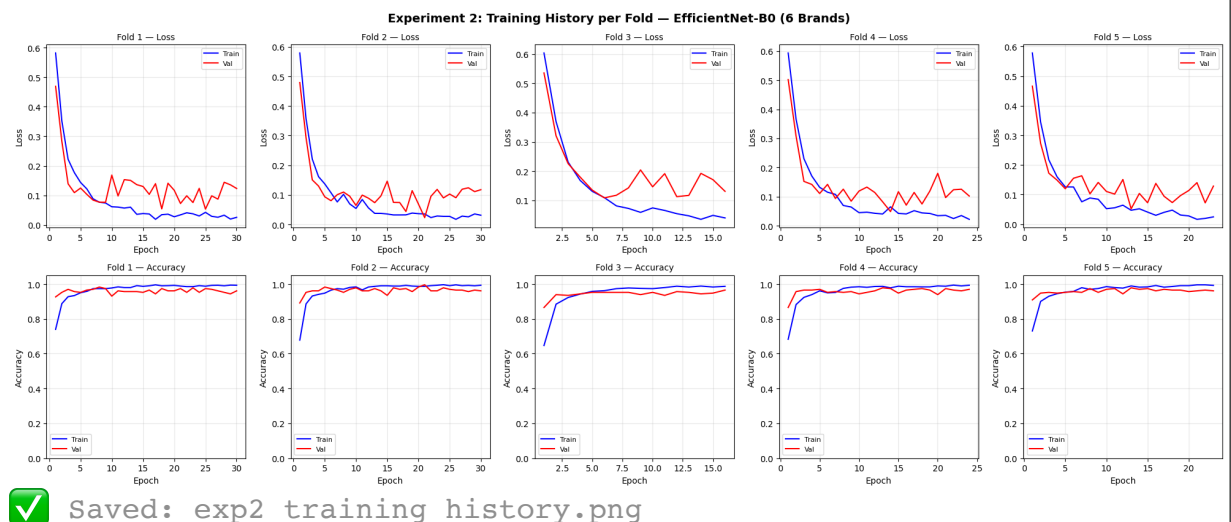
for fold_idx, history in enumerate(fold_histories):
    ax_loss = axes[0, fold_idx]
    ax_acc = axes[1, fold_idx]
    epochs_range = range(1, len(history['train_loss']) + 1)

    ax_loss.plot(epochs_range, history['train_loss'], 'b-', label='Train Loss')
    ax_loss.plot(epochs_range, history['val_loss'], 'r-', label='Val Loss')
    ax_loss.set_title(f'Fold {fold_idx+1} – Loss', fontsize=10)
    ax_loss.set_xlabel('Epoch'); ax_loss.set_ylabel('Loss')
    ax_loss.legend(fontsize=8); ax_loss.grid(True, alpha=0.3)

    ax_acc.plot(epochs_range, history['train_acc'], 'b-', label='Train Accuracy')
    ax_acc.plot(epochs_range, history['val_acc'], 'r-', label='Val Accuracy')
    ax_acc.set_title(f'Fold {fold_idx+1} – Accuracy', fontsize=10)
    ax_acc.set_xlabel('Epoch'); ax_acc.set_ylabel('Accuracy')
    ax_acc.legend(fontsize=8); ax_acc.grid(True, alpha=0.3)
    ax_acc.set_ylim(0, 1.05)

plt.tight_layout()
plt.savefig('exp2_training_history.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: exp2_training_history.png")

```



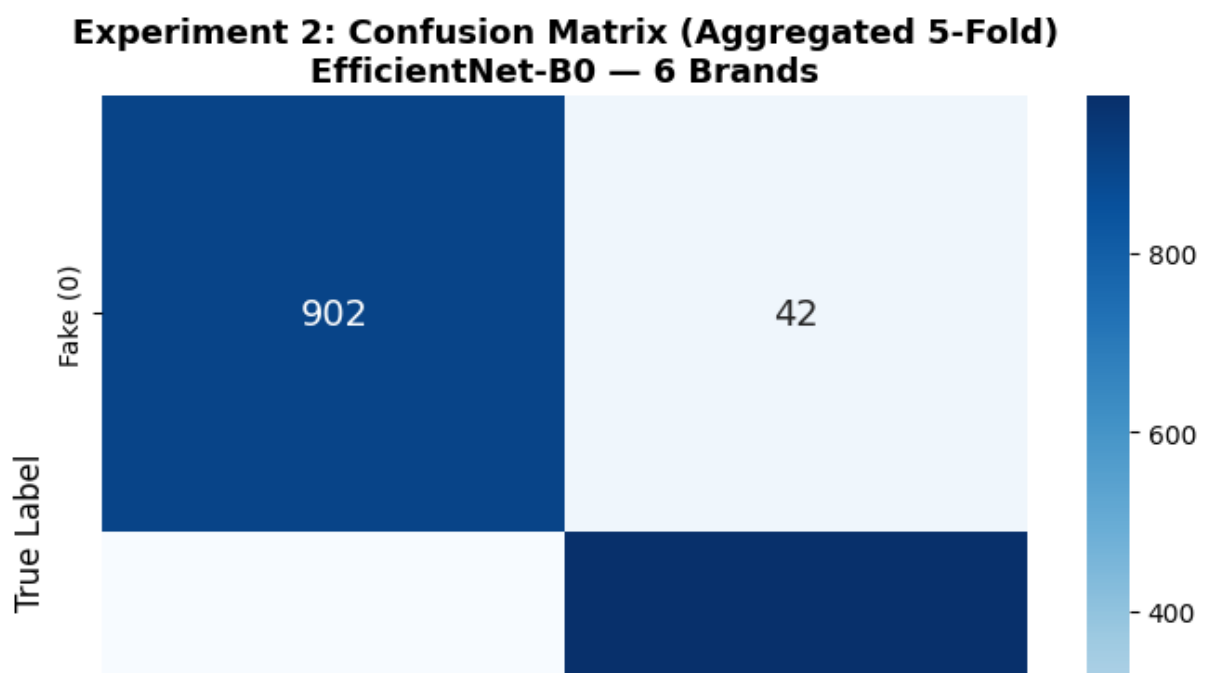
```
# =====
# 9 الخليّة : Confusion Matrix
# =====

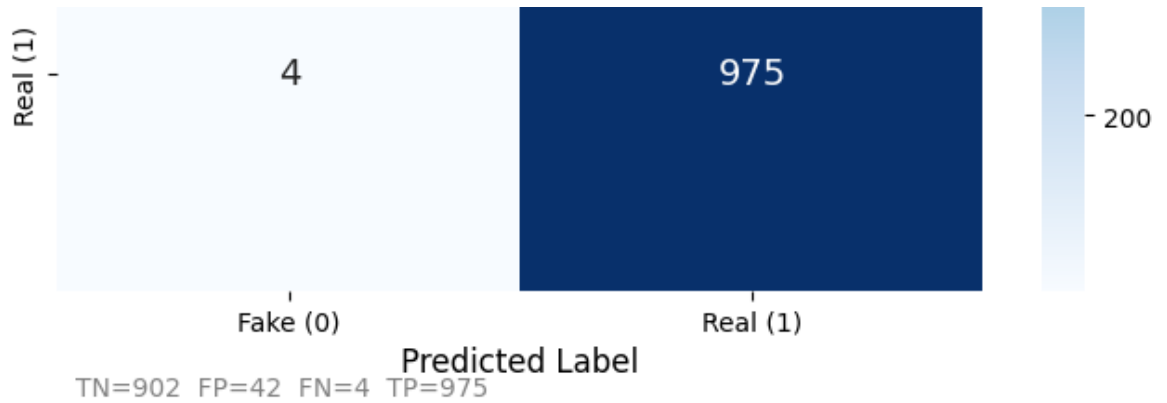
cm = confusion_matrix(all_true, all_pred)

fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=['Fake (0)', 'Real (1)'],
             yticklabels=['Fake (0)', 'Real (1)'],
             annot_kws={'size': 14})
ax.set_xlabel('Predicted Label', fontsize=12)
ax.set_ylabel('True Label', fontsize=12)
ax.set_title('Experiment 2: Confusion Matrix (Aggregated 5-Fold)')

tn, fp, fn, tp = cm.ravel()
ax.text(0.02, -0.12, f'TN={tn}  FP={fp}  FN={fn}  TP={tp}', transform=ax.transAxes)

plt.tight_layout()
plt.savefig('exp2_confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: exp2_confusion_matrix.png")
```





✓ Saved: exp2\_confusion\_matrix.png

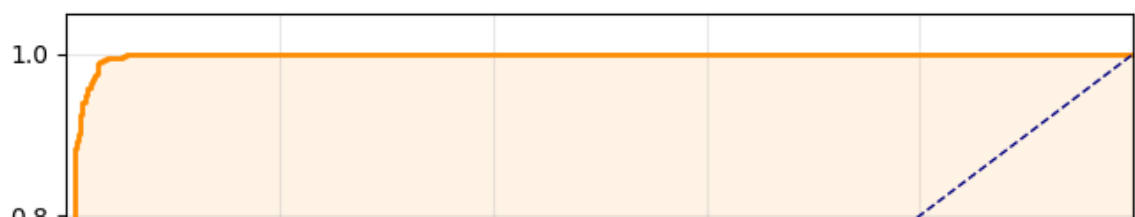
```
# =====
# 10 الخلية : ROC Curve
# =====

fpr, tpr, _ = roc_curve(all_true, all_prob)
auc_score = roc_auc_score(all_true, all_prob)

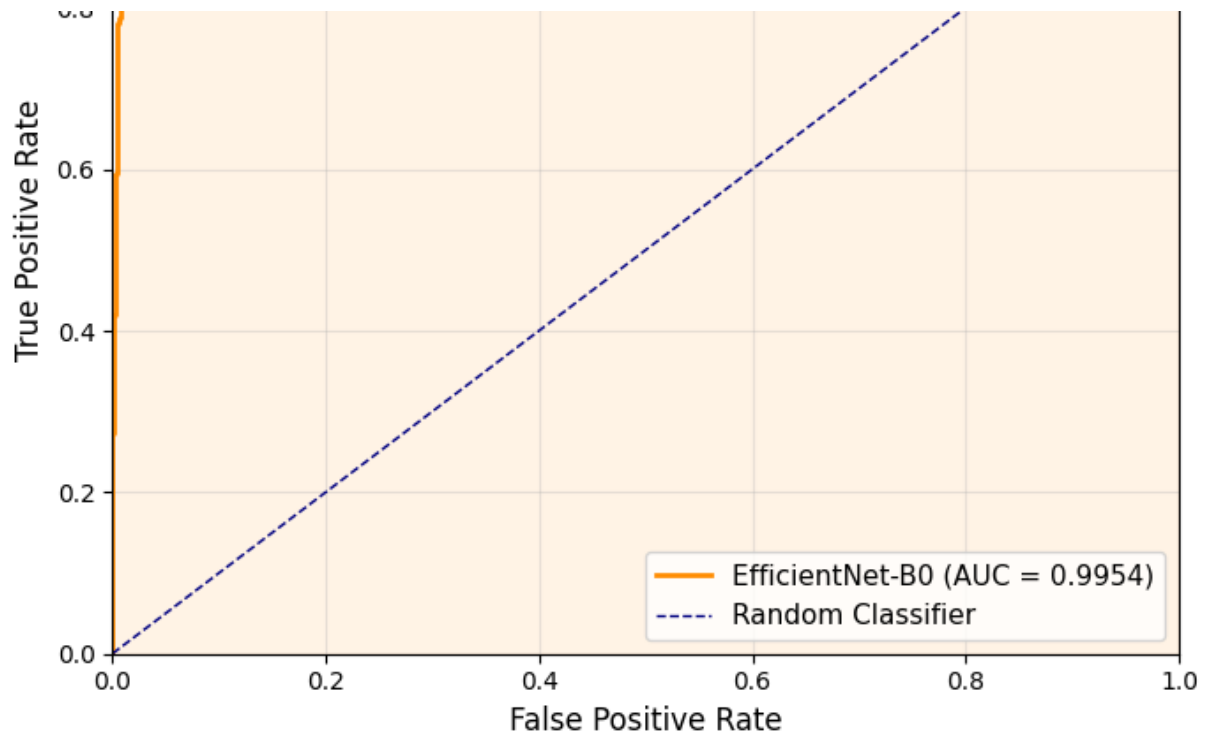
fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr, tpr, color='darkorange', lw=2, label=f'EfficientNet')
ax.plot([0, 1], [0, 1], color='navy', lw=1, linestyle='--', label='Random')
ax.fill_between(fpr, tpr, alpha=0.1, color='darkorange')
ax.set_xlim([0.0, 1.0]); ax.set_ylim([0.0, 1.05])
ax.set_xlabel('False Positive Rate', fontsize=12)
ax.set_ylabel('True Positive Rate', fontsize=12)
ax.set_title('Experiment 2: ROC Curve (Aggregated 5-Fold)\nEfficientNet-B0 — 6 Brands')
ax.legend(loc='lower right', fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('exp2_roc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
print("✓ Saved: exp2_roc_curve.png")
```

**Experiment 2: ROC Curve (Aggregated 5-Fold)**  
**EfficientNet-B0 — 6 Brands**







✓ Saved: exp2\_roc\_curve.png